

به نام خدا

سیستم تحلیل لاگ

۱. مقدمه

در این پروژه هدف ما طراحی و پیاده‌سازی یک سیستم مقیاس‌پذیر برای تحلیل لاگ است که قابلیت دریافت، ذخیره‌سازی، جستجو و مرور ایونت‌ها را داشته باشد. طراحی سیستم باید به گونه‌ای باشد که با ورود میلیون‌ها لاگ در روز بتواند عملکرد قابل قبولی ارائه دهد و از ابزارهای مدرن توزیع‌شده نظیر Kafka، Cassandra، ClickHouse و CockroachDB به درستی بهره ببرد.

۲. معماری سیستم

در این بخش سیستم به مؤلفه‌های زیر تقسیم می‌شود:

- Web UI (فرانت‌اند): ساخت پروژه، نمایش اطلاعات پروژه‌ها، ورود کاربر، جستجوی ایونت‌ها و مشاهده

جزئیات

- Backend API: نوشته‌شده با Go، پیاده‌سازی endpoint های RESTful، احراز هویت، ثبت لاگ
- Kafka Producer: برای queue کردن لاگ‌های ورودی به شکل غیرهمزمان
- Kafka Consumer: پردازش لاگ‌های دریافتی و ارسال به دیتابیس‌ها
- Cassandra: ذخیره‌سازی اصلی ایونت‌ها با TTL و ساختار مناسب پارتیشن‌بندی
- ClickHouse: ذخیره‌سازی تحلیلی ایونت‌ها برای query های سریع و آماری
- CockroachDB: مدیریت کاربران، پروژه‌ها و API Key

جریان داده به صورت زیر است:

۱. کاربر از طریق UI یا CLI یک پروژه ایجاد می‌کند. متادیتا پروژه در CockroachDB ذخیره می‌شود.
۲. به پروژه یک api_key اختصاص داده می‌شود.
۳. برنامه‌نویسان با api_key لاگ‌هایی را به REST API ارسال می‌کنند.
۴. API لاگ‌ها را به Kafka می‌فرستد.

۵. Consumer لاگ‌ها را از Kafka دریافت می‌کند و:

* در Cassandra ذخیره می‌کند (برای TTL و دسترسی اولیه)؛

* در ClickHouse ذخیره می‌کند (برای جستجوی آماری)؛

۶. UI با API در تماس است و برای تحلیل، از ClickHouse کوئری می‌گیرد.

۳. دلایل انتخاب ابزارها

Kafka:

الف) مزایا:

* throughput بالا برای پردازش ایونت‌های زیاد؛

* مقیاس‌پذیری افقی و پایدار بودن پیام‌ها؛

* fault-tolerant بودن.

ب) معایب:

* مدیریت offset و consumer lag ممکن است پیچیده شود.

* آلترناتیوی که به جای آن می‌شد استفاده کرد RabbitMQ بود که به دلیل مقیاس‌پذیر نبودن و مناسب

نبودن برای حجم زیاد لاگ‌ها، کافکا انتخاب بهتری بود.

Cassandra:

الف) مزایا:

* write-heavy workload مناسب؛

* native TTL؛

* مقیاس‌پذیر و مقاوم در برابر خطا.

ب) معایب:

* نیاز به طراحی دقیق partition key و clustering key؛

* eventual consistency؛

* دشوار بودن کلاستر کردن آن به دلیل مصرف زیاد رم.

توضیح مشکلات حین کار با کاساندرا

یکی از بزرگ‌ترین چالش‌هایی که در پیاده‌سازی این تمرین داشتیم، بالا آوردن کاساندرا در سه نود بود، بدون اینکه بعد از دو دقیقه پایین بیاید. برای این کار مجبور شدیم محدودیت‌هایی روی مصرف رمش قرار بدیم که روی پرفورمنس پروژه تاثیر منفی داشت. به جای این کار میتونستیم به دیپلوی کردنش توی یک یا دو نود اکتفا کنیم ولی ترجیح دادیم سه‌نوده باشه.

: ClickHouse

مزایا:

* عملکرد عالی در query و aggregation روی میلیون‌ها رکورد؛

* کمترین latency برای تحلیل داده‌ها؛

* احتمالا تنها ابزاری که طی پیاده‌سازی پروژه مشکل خاصی باهاش نداشتیم و از اول درست کارشو می‌کرد

Clickhouse بود.

: CockroachDB

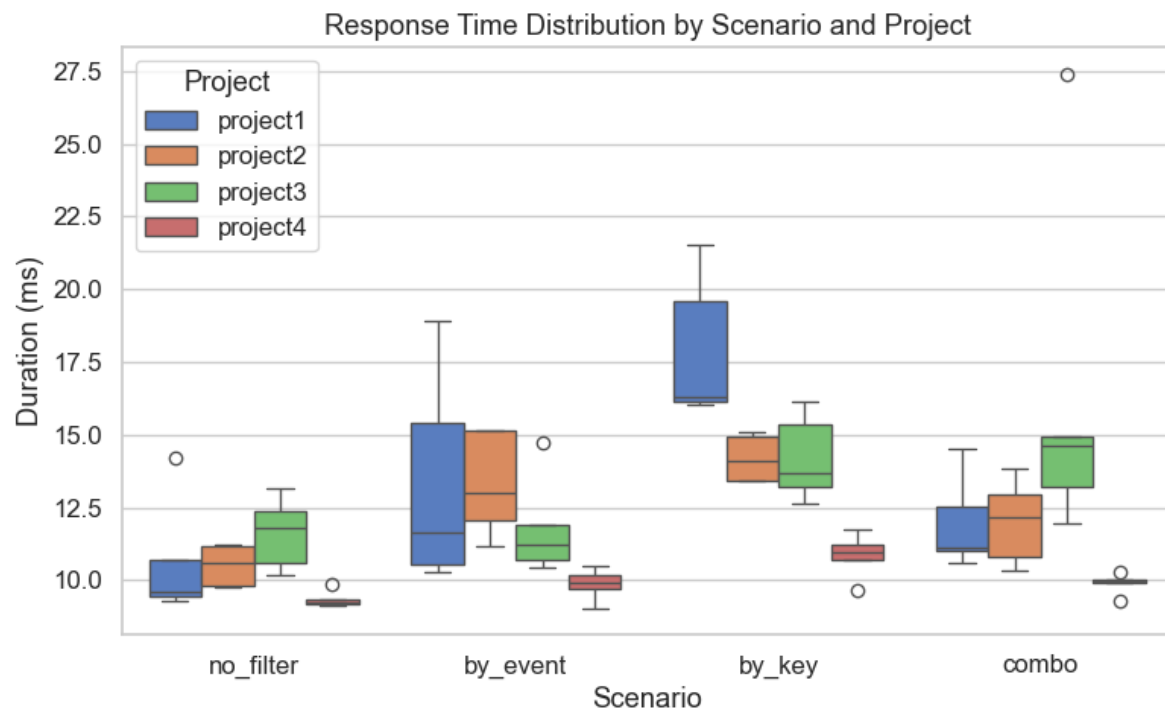
مزایا:

* PostgreSQL-compatible با replication داخلی؛

* مقیاس‌پذیری بالا؛

* کلاستر شدن ساده و بی‌دردسر.

۴. تست عملکرد و تحلیل



نمودار بالا نتیجه‌ی تست ریسپانس تایم پروژه در جستجوی ایونت‌ها در شرایط مختلف است. ابزارها و فایل‌های بیشتر برای تست در cmd/loadTest موجودند.

اعضای گروه:

❖ عرفان قربانی (۴۰۲۱۷۰۲۱۶)

❖ مهدی رسول‌زاده (۴۰۲۱۷۰۱۲۱)

❖ سروش نجفی (۴۰۲۱۰۰۵۵۹)

مرداد ۱۴۰۴